SIMATS ENGINEERING
ASSESSMENT TOOL 2

Student Name	R. Kishan
Register No.	192525378
Course Name	Artificial Intelligence
Course Code	CSA17
Course Outcome	CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively (BL3)
Annexure	Annexure A - Constraint-Based Problem Solving
Questions / Marks	3 Questions / 30 Marks (Duration: 1 Hour)
Weightage	50%

Code of Conduct:

I R. Kishan / 192525378 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Question 1: Doctor-Shift Assignment (Backtracking Search)

CSP formulation

- **Variables:** D1, D2, D3 (one per doctor).
- **Domain of each variable:** {Morning (M), Afternoon (A), Night (N)}, ordered M < A < N.
- **Constraints:** C1: D1 is not Night. C2: D2 works before D3 (shift(D2) < shift(D3)). C3: all-different (one doctor per shift, each doctor exactly one shift - this covers C-iii and C-v). C4: D3 is not Morning.
- **Search order:** variables D1, D2, D3; values tried in the order M, A, N. After every assignment all constraints that involve only assigned variables are checked; on a violation the algorithm backtracks (tries the next value, or goes back to the previous variable).

Step-by-step backtracking trace

Step	Assignment tried	Constraint checks	Result
1	Start: { }	No variable assigned yet	Select D1
2	D1 = M	C1: M is not Night - OK. C3: no clash - OK	Consistent. Go deeper, select D2
3	D1 = M, D2 = M	C3: Morning already taken by D1 - VIOLATED	Reject. BACKTRACK to next value of D2
4	D1 = M, D2 = A	C3: OK. C2: D2 = A can still be before D3 (D3 = N possible) - OK	Consistent. Go deeper, select D3
5	D1 = M, D2 = A, D3 = M	C4: D3 cannot be Morning - VIOLATED. C3: Morning taken - VIOLATED	Reject. Try next value of D3
6	D1 = M, D2 = A, D3 = A	C3: Afternoon taken by D2 - VIOLATED. C2: A is not before A - VIOLATED	Reject. Try next value of D3
7	D1 = M, D2 = A, D3 = N	C1: OK. C2: A < N - OK. C3: M, A, N all different - OK. C4: Night is not Morning - OK. C5: all 3 doctors assigned - OK	**All constraints satisfied. GOAL.**

Search tree (depth-first):

Root -> D1=M -> { D2=M (X, C3) ; D2=A -> { D3=M (X, C4/C3) ; D3=A (X, C3/C2) ; D3=N (OK - solution) } }

Doctor	Shift
D1	Morning
D2	Afternoon
D3	Night

Final valid schedule: D1 - Morning, D2 - Afternoon, D3 - Night.

Note: if the search is continued, the only other solution is D1 = Afternoon, D2 = Morning, D3 = Night. Backtracking stops at the first solution found, which is the one above. (D1 = Night would have been rejected immediately by C1.)

Question 2: Grid Robot - Priority-Queue Search with Manhattan Heuristic

Problem set-up

The question does not include a picture of the 5x5 grid, so the following grid is **assumed** (cells are written as (row, column), rows 0-4 from top, columns 0-4 from left). Start S = (2,0), Goal G = (2,4), obstacles = (1,2), (2,2), (3,2). The same method works for any other grid.

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	(1,1)	**X** (1,2)	(1,3)	(1,4)
S (2,0)	(2,1)	**X** (2,2)	(2,3)	**G** (2,4)
(3,0)	(3,1)	**X** (3,2)	(3,3)	(3,4)
(4,0)	(4,1)	(4,2)	(4,3)	(4,4)

Green = Start, Red = Goal, Grey X = obstacle, Yellow = optimal path found.

Method

- **Moves:** Up, Down, Left, Right; each move costs 1, so g(n) = number of moves from S. Obstacles are never expanded.
- **Heuristic:** h(n) = |row - 2| + |col - 4| (Manhattan distance to G). It never over-estimates because each move changes only one coordinate by 1, so it is admissible and consistent.
- **Priority queue (OPEN):** BFS explores level by level; to use the Manhattan heuristic the FIFO queue is replaced by a priority queue ordered by f(n) = g(n) + h(n) (best-first / A* style expansion of the BFS tree). Ties are broken by smaller h, then by insertion order (neighbours generated in order U, D, L, R). Already-expanded nodes (CLOSED) are not re-added.

Step-by-step node expansion and priority-queue updates

#	Node expanded (g, h, f)	Children added to OPEN (g)	OPEN after step: node(f) in priority order
1	**(2,0)** g=0, h=4, f=4	(1,0) g=1, (3,0) g=1, (2,1) g=1	(2,1):4, (1,0):6, (3,0):6
2	**(2,1)** g=1, h=3, f=4	(1,1) g=2, (3,1) g=2	(1,1):6, (3,1):6, (1,0):6, (3,0):6
3	**(1,1)** g=2, h=4, f=6	(0,1) g=3	(3,1):6, (1,0):6, (3,0):6, (0,1):8
4	**(3,1)** g=2, h=4, f=6	(4,1) g=3	(1,0):6, (3,0):6, (0,1):8, (4,1):8
5	**(1,0)** g=1, h=5, f=6	(0,0) g=2	(3,0):6, (0,1):8, (4,1):8, (0,0):8
6	**(3,0)** g=1, h=5, f=6	(4,0) g=2	(0,1):8, (4,1):8, (0,0):8, (4,0):8
7	**(0,1)** g=3, h=5, f=8	(0,2) g=4	(0,2):8, (4,1):8, (0,0):8, (4,0):8
8	**(0,2)** g=4, h=4, f=8	(0,3) g=5	(0,3):8, (4,1):8, (0,0):8, (4,0):8
9	**(0,3)** g=5, h=3, f=8	(1,3) g=6, (0,4) g=6	(1,3):8, (0,4):8, (4,1):8, (0,0):8, (4,0):8
10	**(1,3)** g=6, h=2, f=8	(2,3) g=7, (1,4) g=7	(2,3):8, (1,4):8, (0,4):8, (4,1):8, (0,0):8, (4,0):8
11	**(2,3)** g=7, h=1, f=8	(3,3) g=8, (2,4) g=8	(2,4):8, (1,4):8, (0,4):8, (4,1):8, (0,0):8, (4,0):8, (3,3):10
12	**(2,4)** g=8, h=0, f=8	-- GOAL TEST SUCCESS --	-

Initial OPEN = [(2,0):4]. The search expanded 12 nodes; the nodes (1,0), (3,0) are expanded at f = 6 and the detour around the wall is then found through the top route at f = 8. The bottom route (e.g. (4,1), (4,0)) is left in OPEN because the top route reaches G first with the same f = 8.

Optimal path: (2,0) -> (2,1) -> (1,1) -> (0,1) -> (0,2) -> (0,3) -> (1,3) -> (2,3) -> (2,4)
Path cost = 8 (8 moves x cost 1). It is optimal because h is admissible: no path can be shorter than 4 (Manhattan distance), and the wall (1,2)-(3,2) forces 2 extra vertical moves up/down and 2 back, giving 8. An equally short mirror path exists through row 4.
Remark: with unit costs, plain BFS also gives the same optimal cost 8, but it would expand more nodes because it ignores the goal direction; the heuristic-guided priority queue reduces the search effort.

Question 3: Autonomous Rescue Robot (Partially Observable Environment)

(a) Analysis of constraints and their effect on decision-making

Constraint	Effect on the robot's decisions	Priority
Blocked cells (obstacles)	Remove those cells from the state space; only valid moves are generated. A plan may become invalid when a new blockage is sensed, so replanning is needed.	1 - Hard
Limited sensing (adjacent cells only)	Environment is partially observable. The robot cannot plan on a complete map; it keeps a belief/map (known free, known blocked, unknown) and decides using what it has seen so far.	1 - Hard
Safe navigation	Moves into blocked or unsafe cells are not allowed; risky cells are entered only if no cheaper safe alternative exists.	1 - Hard
Moves: Up/Down/Left/Right	Branching factor at most 4; defines the successor function; diagonal shortcuts are not allowed.	2 - Structural
Cost: 1 per move, +2 for a risky zone (entering a risky cell costs 3)	Costs are not uniform, so BFS is not optimal; a cost-based search (UCS / A*) is needed. A short path through 2 risky cells can be worse than a longer safe path.	3 - Optimisation
Minimise total cost	Objective function: pick the least-cost path to G among all safe paths.	3 - Optimisation
Update knowledge dynamically (online search)	Planning and acting are interleaved: act, sense, update map, replan. Offline planning once at the start is not enough.	2 - Structural

Prioritisation: feasibility and safety (hard constraints) come first, then the information constraints (sensing/online updating), and cost minimisation is applied only among the feasible, safe options.

(b) Solution approach: Online Search Agent using Uniform Cost Search (A*) replanning

Problem formulation: State = (robot cell, known map). Actions = U, D, L, R. Transition cost = 1 for a normal cell, 3 for a risky cell. Goal = cell G. Path cost = sum of step costs.

Steps of the algorithm:

1. **Initialise:** robot at S; the known map is empty except S. Unknown cells are optimistically treated as free (cost 1), so that the plan is never longer than the truth.
2. **Sense:** observe the four adjacent cells; mark each as free, blocked or risky and store it in the map.
3. **Plan:** run Uniform Cost Search (or A* with Manhattan distance as an admissible heuristic) on the current map from the robot cell to G. g(n) is the accumulated cost including the +2 penalties. The priority queue always expands the lowest-cost node.
4. **Act:** execute only the first move of the best plan (never enter a cell known to be blocked; avoid a risky cell if an equal or cheaper safe route exists).

5. **Update and replan:** after the move, sense again. If a new obstacle or risky zone changes the plan, discard the old plan and go to step 3. Keep a visited list to avoid loops.
6. **Terminate:** stop when the robot reaches G (survivor located). If the open list becomes empty, G is unreachable with the current knowledge, so the robot backtracks to the nearest unexplored frontier cell and explores from there.

Small example of cost handling: Route A has 6 moves and passes through 2 risky cells: cost = 6 + 2x2 = 10. Route B is a safe detour of 8 moves: cost = 8. UCS expands Route B first and the robot chooses it, so the total cost is minimised while staying safe. If Route B later turns out to be blocked, the robot senses it, updates the map and replans, possibly choosing Route A.

(c) Application of AI concepts

Concept	How it applies to the rescue robot
Intelligent agent (PEAS)	**Performance:** reach G with minimum cost, safely. **Environment:** disaster-hit building grid. **Actuators:** wheels/move commands. **Sensors:** adjacent-cell sensors. The agent is a **model-based, utility/goal-based agent**: it keeps an internal map (model) and chooses actions that reach the goal with the least cost.
Rationality	A rational agent selects the action expected to maximise its performance measure given its percepts and knowledge. Here: at each step the robot chooses the move on the lowest-cost known safe path, gathers information as it moves, and replans. It does not need to be omniscient - only to use available information well.
Environment type	**Partially observable** (adjacent cells only); **single agent**; **deterministic** (a move goes where intended, assumption); **sequential** (each move affects later cost); **static / semi-dynamic** (blocked cells can change as debris moves - handled by replanning); **discrete** (grid cells); **unknown** (map is discovered while moving).

(d) Justification of the chosen approach

- **Non-uniform costs:** BFS ignores the +2 risk penalty and can return a cheap-looking but costly path; UCS/A* always returns the least-cost path (complete and optimal since every step cost is at least 1).
- **Partial observability:** an offline plan would fail when unseen obstacles appear; an online agent that senses, updates the map and replans copes with this naturally.
- **Safety:** blocked cells are removed from the search and risky cells carry a penalty, so the least-cost solution is also the safest practical one.
- **Efficiency:** adding the Manhattan-distance heuristic (admissible, because every step costs at least 1) keeps optimality while expanding fewer nodes than plain UCS. Depth-first or greedy search could be trapped in dead ends or give costly paths.
- **Rationality:** the approach uses all current percepts and knowledge to minimise cost at every step, which matches the definition of a rational agent.

Conclusion: an online Uniform Cost Search / A* replanning agent is the most suitable strategy because it handles unequal costs, hidden information and safety requirements together.